Project 2 Secure Social Media Applications

William Sunderland 23345180

Project Description

My project has created a secure twitter clone that utilises encryption so that only users within a group can see posts. The app has a registration and login system. Once logged in the user can create a group and add other users to this group. The user can leave groups and the group owner can delete the group or remove members. The user can post to the global feed on the right. They must input what group they are posting to. The post is encrypted through a group key specific to every group. Only this encrypted data is present on the server. If you are not in the group you can only read encrypted data. If you are in the group you can read the decrypted post which is decrypted client side. If you leave the group the keys are rotated across the other members so as not to leave a security issue and you will no longer be able to read the group's posts.

High Level Design

The project is dockerised and split into four distinct parts:

- Frontend - React
- App-server - Python
- Key-server - Python
- Postgres Database

Each part can communicate through fast API endpoints set up to pass relevant data between.

Frontend

- Handles the users encryption/decryption
- Stores private keys locally
- Signs posts
- Verifies certificates and signatures
- Displays the UI the user can interact with

App-server

- Stores posts as encryptions
- Manages users, groups, memberships

Key-server

- Issues certificates
- Manages group keys
- Handles key distribution and rotation

Database

- Stores users
- Stores posts
- Stores keys
- Stores certificates

Encryption

I aimed to follow the same architecture of TLS 1.3 as TLS 1.3 is part of this course. I used ECC(ECDSA and ECCDH), HKDF and AESGCM as my encryption protocols I used the built in crypto.subtle in javascript and the cryptography library in python to implement encryption.

Certificate Authority

The key-server establishes itself as the Certificate Authority on its first run and will store this to ensure all certificates issued are still valid. If the database is emptied it will reestablish itself.

User Key Generation

- When the user registers or every time the user logs in two ECC key pairs are generated.
- An ECDSA key pair for signing posts to prove authenticity and an ECDH key pair used for key agreement in the exchange of the group keys are generated.
- The key server will issue an X.509 certificate for the signing public key using the Certificate Authority
- The user stores the private key locally and registers the certificate with the app server.

Group Keys

- Each Group has a symmetric AES key used to encrypt and decrypt all the posts within the group.
- An initial group key is derived when the group key is created and its number is set to 1.
- If the key is rotated a new key is generated and its identification number is updated.
- The key is wrapped through ECDH, HKDF and then AES to encrypt this uniquely for each member of the group.
- These keys can then be distributed to each member when required.

Posting

- In order to post, the user requests the wrapped group key from the server.
- It then unwraps the group key locally using its ECDH private jey in combination with the key servers public key, HKDF and AES-GCM.
- It then uses this group key to encrypt the post using AES_GCM.
- The encrypted payload is signed by the user using ECDSA
- The app-server stores the cipher text

Reading

- In the same vein as posting in order to read posts, the user requests the wrapped group key from the server.

- It then unwraps the group key locally using its ECDH private jey in combination with the key servers public key, HKDF and AES-GCM.
- It then uses this group key to decrypt the post using AES_GCM.
- The user verifies the signature using the author's ECDSA public key.

Code

I have only attached the code that is specific to the cryptography as otherwise the pdf would be excessively long due to a lot of frontend and database management code that isn't relevant to the project.

frontend

frontend/src/crypto.js

```
const encoder = new TextEncoder();
const decoder = new TextDecoder();

//convert uint8array to base64 string
function toBase64(bytes) {
  let binary = "";
  bytes.forEach((b) => (binary += String.fromCharCode(b)));
  return btoa(binary);
}
//normalise base64
function normalizeBase64(base64) {
  const normalized = base64.replace(/-/g, "+").replace(/_/g, "/");
  const pad = normalized.length % 4 === 0 ? "" : "=".repeat(4 -
(normalized.length % 4));
  return normalized + pad;
}

//convert from base64 string to uint8array
function fromBase64(base64) {
  const binary = atob(normalizeBase64(base64));
  const bytes = new Uint8Array(binary.length);
  for (let i = 0; i < binary.length; i += 1) {
    bytes[i] = binary.charCodeAt(i);
  }
  return bytes;
}

//wrap key bytes into PEM format
function wrapPem(label, bytes) {
  const base64 = toBase64(new Uint8Array(bytes));
  const lines = base64.match(/.{1,64}/g) || [];
  return `-----BEGIN ${label}-----\n${lines.join("\n")}\n-----END
${label}-----`;
}

//extract key bytes from PEM format
```

```
function unwrapPem(pem) {
  const lines = pem.trim().split(/\r?\n/);
  const base64 = lines.filter((line) => !line.startsWith("-----
")).join("");
  return fromBase64(base64).buffer;
}

//Generate ECDSA and ECDH key pairs for the user
export async function generateUserKeys() {
  const signing = await crypto.subtle.generateKey(
    {name: "ECDSA", namedCurve: "P-256"},
    true,
    ["sign", "verify"]
  );
  const agreement = await crypto.subtle.generateKey(
    {name: "ECDH", namedCurve: "P-256"},
    true,
    ["deriveBits"]
  );
  return { signing, agreement };
}

//export a public key to PEM
export async function exportPublicKeyPem(key) {
  const spki = await crypto.subtle.exportKey("spki", key);
  return wrapPem("PUBLIC KEY", spki);
}

//export a private key to PEM
export async function exportPrivateKeyPem(key) {
  const pkcs8 = await crypto.subtle.exportKey("pkcs8", key);
  return wrapPem("PRIVATE KEY", pkcs8);
}

//import Pem to a public key
export async function importPublicKeyPem(pem, usage) {
  const spki = unwrapPem(pem);
  if (usage === "verify") {
      return crypto.subtle.importKey("spki", spki, {name: "ECDSA",
namedCurve: "P-256"}, true,["verify"]
    );
  }
  if (usage === "derive") {
    return crypto.subtle.importKey("spki", spki, {name: "ECDH",
namedCurve: "P-256"}, true, []);
  }
  throw new Error("Unknown usage for public key");
}

//import Pem to a private key
export async function importPrivateKeyPem(pem, usage) {
  const pkcs8 = unwrapPem(pem);
  if (usage === "sign") {
    return crypto.subtle.importKey("pkcs8", pkcs8, {name: "ECDSA",
```

```
namedCurve: "P-256"}, true, ["sign"]);
  }
  if (usage === "derive") {
    return crypto.subtle.importKey("pkcs8", pkcs8, {name: "ECDH",
namedCurve: "P-256"}, true, ["deriveBits"]);
  }
  throw new Error("Unknown usage for private key");
}

//sign a string with ECDSA and return a signature
export async function signMessage(privateKey, message) {
  const data = typeof message === "string" ? encoder.encode(message) :
message;
  const sig = await crypto.subtle.sign({name: "ECDSA", hash: "SHA-256"},
privateKey, data);
  return toBase64(new Uint8Array(sig));
}

//verify ECDSA signature
export async function verifyMessage(publicKey, message, signatureBase64) {
  const data = typeof message === "string" ? encoder.encode(message) :
message;
  const sig = fromBase64(signatureBase64);
  return crypto.subtle.verify({name: "ECDSA", hash: "SHA-256"}, publicKey,
sig, data);
}

//generate a group key
export function generateGroupKey() {
  const key = new Uint8Array(32);
  crypto.getRandomValues(key);
  return key;
}

//create AES-GCM key for wrapping from ECDH
async function deriveWrapKey(sharedSecret, context) {
  const keyMaterial = await crypto.subtle.importKey("raw", sharedSecret,
{name: "HKDF"}, false,["deriveKey"]);
  return crypto.subtle.deriveKey(
    {name: "HKDF", hash: "SHA-256", salt: new Uint8Array(0), info:
encoder.encode(context)},
    keyMaterial,
    {name: "AES-GCM", length: 256},
    false,
    ["encrypt", "decrypt"]
  );
}

//wrap groupkey using ECDH + HKDF + AES
export async function wrapGroupKey(groupKeyBytes,
userAgreementPublicKeyPem, context) {
  const userPublicKey = await
importPublicKeyPem(userAgreementPublicKeyPem, "derive");
  const ephemeral = await crypto.subtle.generateKey(
```

```
    {name: "ECDH", namedCurve: "P-256"},
    true,
    ["deriveBits"]
  );
  const sharedSecret = await crypto.subtle.deriveBits(
    {name: "ECDH", public: userPublicKey},
    ephemeral.privateKey,
    256
  );
  const wrapKey = await deriveWrapKey(sharedSecret, context);
  const nonce = crypto.getRandomValues(new Uint8Array(12));
  const ciphertext = await crypto.subtle.encrypt(
    {name: "AES-GCM", iv: nonce, additionalData: encoder.encode(context)},
    wrapKey,
    groupKeyBytes
  );
  const ephPubPem = await exportPublicKeyPem(ephemeral.publicKey);
  const payload = {ephemeral_pub_key_pem: ephPubPem, nonce:
toBase64(nonce), ciphertext: toBase64(new Uint8Array(ciphertext))};
  return toBase64(encoder.encode(JSON.stringify(payload)));
}

//unwrap GroupKey
export async function unwrapGroupKey(wrapped, userAgreementPrivateKeyPem,
context) {
  const payloadJson = decoder.decode(fromBase64(wrapped));
  const payload = JSON.parse(payloadJson);
  const ephPub = await importPublicKeyPem(payload.ephemeral_pub_key_pem,
"derive");
  const userPriv = await importPrivateKeyPem(userAgreementPrivateKeyPem,
"derive");
  const sharedSecret = await crypto.subtle.deriveBits(
    {name: "ECDH", public: ephPub},
    userPriv,
    256
  );
  const wrapKey = await deriveWrapKey(sharedSecret, context);
  const nonce = fromBase64(payload.nonce);
  const ciphertext = fromBase64(payload.ciphertext);
  const plain = await crypto.subtle.decrypt(
    {name: "AES-GCM", iv: nonce, additionalData: encoder.encode(context)},
    wrapKey,
    ciphertext
  );
  return new Uint8Array(plain);
}

//encrypts the post with the group key
export async function encryptPost(groupKeyBytes, plaintext, aad = "") {
  const key = await crypto.subtle.importKey("raw", groupKeyBytes, {name:
"AES-GCM"}, false, ["encrypt"]);
  const iv = crypto.getRandomValues(new Uint8Array(12));
  const data = typeof plaintext === "string" ? encoder.encode(plaintext) :
plaintext;
```

```
  const cipher = await crypto.subtle.encrypt(
    {name: "AES-GCM", iv, additionalData: encoder.encode(aad)},
    key,
    data
  );
  return {iv: toBase64(iv), ciphertext: toBase64(new Uint8Array(cipher)),
aad};
}

//decrypts the post
export async function decryptPost(groupKeyBytes, ivBase64,
ciphertextBase64, aad = "") {
  const key = await crypto.subtle.importKey("raw", groupKeyBytes, {name:
"AES-GCM"}, false, ["decrypt"]);
  const iv = fromBase64(ivBase64);
  const ciphertext = fromBase64(ciphertextBase64);
  const plain = await crypto.subtle.decrypt(
    {name: "AES-GCM", iv, additionalData: encoder.encode(aad)},
    key,
    ciphertext
  );
  return decoder.decode(plain);
}
```

keyserver

keyserver/app/crypto.py

```
from __future__ import annotations
import base64
import json
import os
from dataclasses import dataclass
from datetime import datetime, timedelta, timezone
from cryptography import x509
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.x509.oid import NameOID

#hold CA private key
@dataclass
class CAKeys:
    private_key: ec.EllipticCurvePrivateKey
    certificate_pem: str

#encode bytes as a string
def _b64(data: bytes) -> str:
    return base64.urlsafe_b64encode(data).decode("utf-8")
```

```
#decode string back into bytes
def _b64d(data: str) -> bytes:
    return base64.urlsafe_b64decode(data.encode("utf-8"))

#create a certificate authority to sign all the users certificates
def create_ca() -> CAKeys:
    private_key = ec.generate_private_key(ec.SECP256R1())
    subject = issuer = x509.Name([
            x509.NameAttribute(NameOID.COUNTRY_NAME, "IE"),
            x509.NameAttribute(NameOID.ORGANIZATION_NAME, "SecureMedia"),
            x509.NameAttribute(NameOID.COMMON_NAME, "SecureMedia CA")])
    cert = (
        x509.CertificateBuilder()
        .subject_name(subject)
        .issuer_name(issuer)
        .public_key(private_key.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(datetime.now(timezone.utc))
        .not_valid_after(datetime.now(timezone.utc) +
timedelta(days=3650))
        .add_extension(x509.BasicConstraints(ca=True, path_length=None),
critical=True)
        .sign(private_key, hashes.SHA256()))
    return CAKeys(private_key=private_key,
certificate_pem=cert.public_bytes(serialization.Encoding.PEM).decode("utf-
8"))

#serialize the CA private key so that it can be stored in the database and
ensure certificates issued stay valid.
def serialize_private_key(private_key: ec.EllipticCurvePrivateKey) -> str:
    return private_key.private_bytes(
        serialization.Encoding.PEM,
        serialization.PrivateFormat.PKCS8,
        serialization.NoEncryption()).decode("utf-8")

#converts the str from the database back into a CA private key
def load_private_key(private_key_pem: str) -> ec.EllipticCurvePrivateKey:
    key = serialization.load_pem_private_key(private_key_pem.encode("utf-
8"), password=None)
    if not isinstance(key, ec.EllipticCurvePrivateKey): raise
ValueError("CA private key must be EC")
    return key

#creates and signs a X.509 certificate for the user
def issue_user_certificate(ca_private_key: ec.EllipticCurvePrivateKey,
user_id: str, username: str, signing_public_key_pem: str,) -> str:
    public_key =
serialization.load_pem_public_key(signing_public_key_pem.encode("utf-8"))
    if not isinstance(public_key, ec.EllipticCurvePublicKey):raise
ValueError("Signing public key must be EC")
    subject = x509.Name([
            x509.NameAttribute(NameOID.ORGANIZATION_NAME, "SecureMedia"),
            x509.NameAttribute(NameOID.COMMON_NAME, username),
```

```
            x509.NameAttribute(NameOID.SERIAL_NUMBER, user_id)])
    cert =
(x509.CertificateBuilder().subject_name(subject).issuer_name(x509.Name(
                [x509.NameAttribute(NameOID.ORGANIZATION_NAME,
"SecureMedia"),x509.NameAttribute(NameOID.COMMON_NAME, "SecureMedia
CA"),]))
        .public_key(public_key)
        .serial_number(x509.random_serial_number())
        .not_valid_before(datetime.now(timezone.utc))
        .not_valid_after(datetime.now(timezone.utc) + timedelta(days=365))
        .sign(ca_private_key, hashes.SHA256())
    )
    return cert.public_bytes(serialization.Encoding.PEM).decode("utf-8")

#create a new AES key to be used as a group key
def generate_group_key() -> bytes:
    return AESGCM.generate_key(bit_length=256)

#derives a wrap key from the ECDH secret
def _derive_wrap_key(shared_secret: bytes, context: bytes) -> bytes:
    return HKDF(algorithm=hashes.SHA256(), length=32, salt=None,
info=context).derive(shared_secret)

#wraps the group key using ECDH + HKDF + AES-GCM
def wrap_group_key(group_key: bytes, user_agreement_public_key_pem: str,
context: bytes,) -> str:
    user_public_key =
serialization.load_pem_public_key(user_agreement_public_key_pem.encode("ut
f-8"))
    if not isinstance(user_public_key, ec.EllipticCurvePublicKey): raise
ValueError("Agreement public key must be EC")
    ephemeral_private = ec.generate_private_key(ec.SECP256R1())
    shared_secret = ephemeral_private.exchange(ec.ECDH(), user_public_key)
    wrap_key = _derive_wrap_key(shared_secret, context)
    nonce = os.urandom(12)
    aesgcm = AESGCM(wrap_key)
    ciphertext = aesgcm.encrypt(nonce, group_key, context)
    eph_pub_pem =
ephemeral_private.public_key().public_bytes(serialization.Encoding.PEM,
serialization.PublicFormat.SubjectPublicKeyInfo)
    payload = {
        "ephemeral_pub_key_pem": eph_pub_pem.decode("utf-8"),
        "nonce": _b64(nonce),
        "ciphertext": _b64(ciphertext),
    }
    return _b64(json.dumps(payload).encode("utf-8"))

#unwrap the group key from the payload
def unwrap_group_key(wrapped: str, user_agreement_private_key:
ec.EllipticCurvePrivateKey, context: bytes) -> bytes:
    payload = json.loads(_b64d(wrapped))
    eph_pub =
serialization.load_pem_public_key(payload["ephemeral_pub_key_pem"].encode(
"utf-8"))
```

```
    shared_secret = user_agreement_private_key.exchange(ec.ECDH(),
eph_pub)
    wrap_key = _derive_wrap_key(shared_secret, context)
    nonce = _b64d(payload["nonce"])
    ciphertext = _b64d(payload["ciphertext"])
    return AESGCM(wrap_key).decrypt(nonce, ciphertext, context)
```

keyserver/app/routes.py

```
from fastapi import APIRouter, HTTPException
from sqlalchemy import select, func
from .schemas import (
    CertificateRequest,
    CertificateResponse,
    GroupKeysCreateRequest,
    GroupKeysCreateResponse,
    PublicKeysResponse,
    WrappedKeyResponse,
)
from .crypto import (
    create_ca,
    generate_group_key,
    issue_user_certificate,
    load_private_key,
    serialize_private_key,
    wrap_group_key,
)
from .db import get_session
from .models import CARecord, Certificate, GroupKey, PublicKeyBundle,
WrappedKey
router = APIRouter()

#issues issues the user a certificate. Creates a new CA if no user exists
@router.post("/certificates/request", response_model=CertificateResponse)
def request_certificate(payload: CertificateRequest):
    with get_session() as session:
        ca = session.get(CARecord, 1)
        if not ca:
            ca_keys = create_ca()
            ca = CARecord(id=1,
private_key_pem=serialize_private_key(ca_keys.private_key),
certificate_pem=ca_keys.certificate_pem)
            session.add(ca)
            session.commit()
        ca_private_key = load_private_key(ca.private_key_pem)
        cert_pem = issue_user_certificate(ca_private_key, payload.user_id,
payload.username, payload.signing_public_key_pem)
        session.merge(Certificate(user_id=payload.user_id,
cert_pem=cert_pem, revoked=False))
        session.merge(
            PublicKeyBundle(
```

```
                user_id=payload.user_id,
                signing_public_key_pem=payload.signing_public_key_pem,
                agreement_public_key_pem=payload.agreement_public_key_pem,
            )
        )
        session.commit()
        return CertificateResponse(user_id=payload.user_id,
cert_pem=cert_pem)

#retrieves the user's certificate from the database
@router.get("/certificates/{user_id}", response_model=CertificateResponse)
def get_certificate(user_id: str):
    with get_session() as session:
        cert = session.get(Certificate, user_id)
        if not cert: raise HTTPException(status_code=404,
detail="Certificate not found")
        return CertificateResponse(
            user_id=user_id,
            cert_pem=cert.cert_pem,
            revoked=cert.revoked,
        )

# retrieve the certificate authority certificate
@router.get("/ca/certificate")
def get_ca_certificate():
    with get_session() as session:
        ca = session.get(CARecord, 1)
        if not ca: raise HTTPException(status_code=404, detail="CA not
initialized")
        return {"certificate_pem": ca.certificate_pem}

# retrieve the users public key
@router.get("/public-keys/{user_id}", response_model=PublicKeysResponse)
def get_public_keys(user_id: str):
    with get_session() as session:
        keys = session.get(PublicKeyBundle, user_id)
        if not keys: raise HTTPException(status_code=404, detail="Public
keys not found")
        return PublicKeysResponse(
            user_id=user_id,
            signing_public_key_pem=keys.signing_public_key_pem,
            agreement_public_key_pem=keys.agreement_public_key_pem,
        )

#creates a group key if none exist, or reuses an existing key
#wrap the group key for every member in the database
@router.post("/groups/{group_id}/keys/create",
response_model=GroupKeysCreateResponse)
def create_group_keys(group_id: str, payload: GroupKeysCreateRequest):
    with get_session() as session:
        max_version =
session.scalar(select(func.max(GroupKey.version)).where(GroupKey.group_id
== group_id))
        if max_version:
```

```
            version = max_version
            group_key =
session.scalar(select(GroupKey.key_bytes).where(GroupKey.group_id ==
group_id, GroupKey.version == version))
            if not group_key: raise HTTPException(status_code=404,
detail="Group key not found")
        else:
            version = 1
            group_key = generate_group_key()
            session.add(GroupKey(group_id=group_id, version=version,
key_bytes=group_key))
        wrapped = {}
        for member_id in payload.member_user_ids:
            pk = session.get(PublicKeyBundle, member_id)
            if not pk:raise HTTPException(status_code=404,
detail=f"Missing agreement key for {member_id}")
            context = f"{group_id}:{version}:{member_id}".encode("utf-8")
            wrapped_key = wrap_group_key(group_key,
pk.agreement_public_key_pem, context)
            existing = session.scalar(
                select(WrappedKey).where(
                    WrappedKey.group_id == group_id,
                    WrappedKey.version == version,
                    WrappedKey.user_id == member_id,
                )
            )
            if existing: existing.wrapped_key = wrapped_key
            else:
                session.add(
                    WrappedKey(group_id=group_id, version=version,
user_id=member_id, wrapped_key=wrapped_key)
                )
            wrapped[member_id] = wrapped_key
        session.commit()
        return GroupKeysCreateResponse(group_id=group_id, version=version,
wrapped_keys=wrapped)

#Generate a new group key, update version number and wrap it for every
current member
@router.post("/groups/{group_id}/keys/rotate",
response_model=GroupKeysCreateResponse)
def rotate_group_keys(group_id: str, payload: GroupKeysCreateRequest):
    with get_session() as session:
        max_version =
session.scalar(select(func.max(GroupKey.version)).where(GroupKey.group_id
== group_id))
        version = (max_version or 0) + 1
        group_key = generate_group_key()
        session.add(GroupKey(group_id=group_id, version=version,
key_bytes=group_key))
        wrapped = {}
        for member_id in payload.member_user_ids:
            pk = session.get(PublicKeyBundle, member_id)
            if not pk:raise HTTPException(status_code=404,
```

```
detail=f"Missing agreement key for {member_id}")
            context = f"{group_id}:{version}:{member_id}".encode("utf-8")
            wrapped_key = wrap_group_key(group_key,
pk.agreement_public_key_pem, context)
            session.add(
                WrappedKey(group_id=group_id, version=version,
user_id=member_id, wrapped_key=wrapped_key)
            )
            wrapped[member_id] = wrapped_key
        session.commit()
        return GroupKeysCreateResponse(group_id=group_id, version=version,
wrapped_keys=wrapped)

#get the current wrapped group key from the user
@router.get("/groups/{group_id}/keys/current",
response_model=WrappedKeyResponse)
def get_current_key(group_id: str, user_id: str):
    with get_session() as session:
        max_version =
session.scalar(select(func.max(GroupKey.version)).where(GroupKey.group_id
== group_id))
        if not max_version: raise HTTPException(status_code=404,
detail="Group key not found")
        group_key = session.scalar(
            select(GroupKey.key_bytes).where(
                GroupKey.group_id == group_id,
                GroupKey.version == max_version,
            )
        )
        pk = session.get(PublicKeyBundle, user_id)
        if not group_key or not pk: raise HTTPException(status_code=404,
detail="Wrapped key not found")
        context = f"{group_id}:{max_version}:{user_id}".encode("utf-8")
        wrapped = wrap_group_key(group_key, pk.agreement_public_key_pem,
context)
        existing = session.scalar(
            select(WrappedKey).where(
                WrappedKey.group_id == group_id,
                WrappedKey.version == max_version,
                WrappedKey.user_id == user_id,
            )
        )
        if existing: existing.wrapped_key = wrapped
        else:
            session.add(
                WrappedKey(group_id=group_id, version=max_version,
user_id=user_id, wrapped_key=wrapped)
            )
        session.commit()
        return WrappedKeyResponse(group_id=group_id, version=max_version,
user_id=user_id, wrapped_key=wrapped)

# get a specific group wrapped key
# used to view old posts before a key got rotated
```

```
@router.get("/groups/{group_id}/keys/{version}/wrapped/{user_id}",
response_model=WrappedKeyResponse)
def get_wrapped_key(group_id: str, version: int, user_id: str):
    with get_session() as session:
        group_key = session.scalar(
            select(GroupKey.key_bytes).where(GroupKey.group_id ==
group_id, GroupKey.version == version)
        )
        pk = session.get(PublicKeyBundle, user_id)
        if not group_key or not pk: raise HTTPException(status_code=404,
detail="Wrapped key not found")
        context = f"{group_id}:{version}:{user_id}".encode("utf-8")
        wrapped = wrap_group_key(group_key, pk.agreement_public_key_pem,
context)
        existing = session.scalar(
            select(WrappedKey).where(
                WrappedKey.group_id == group_id,
                WrappedKey.version == version,
                WrappedKey.user_id == user_id,
            )
        )
        if existing: existing.wrapped_key = wrapped
        else:
            session.add(
                WrappedKey(group_id=group_id, version=version,
user_id=user_id, wrapped_key=wrapped)
            )
        session.commit()
        return WrappedKeyResponse(group_id=group_id, version=version,
user_id=user_id, wrapped_key=wrapped)
```

appserver

appserver/app/routes.py

```
import base64
from datetime import datetime, timedelta, timezone
from fastapi import APIRouter, Depends, HTTPException
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy import delete, select
from .auth import create_access_token, decode_access_token, hash_password,
verify_password
from .db import get_session
from .models import (
    Certificate,
    Group,
    GroupKeyVersion,
    Membership,
    Post,
    User,
```

```
    WrappedKey,
)
from .schemas import (
    CertificateRegisterRequest,
    CertificateRegisterResponse,
    GroupCreateRequest,
    GroupMemberAddRequest,
    GroupMemberInfo,
    GroupListResponse,
    GroupResponse,
    LoginRequest,
    MeResponse,
    PostCreateRequest,
    PostResponse,
    RegisterRequest,
    TokenResponse,
)

router = APIRouter()
security = HTTPBearer()

#decodes a string into bytes
def _b64d(value: str) -> bytes:
    return base64.b64decode(value.encode("utf-8"))

#encodes bytes into a string
def _b64(value: bytes) -> str:
    return base64.b64encode(value).decode("utf-8")

#extracts userid from token
def get_current_user_id(credentials: HTTPAuthorizationCredentials =
Depends(security)) -> int:
    return decode_access_token(credentials.credentials)

#creates a new user and returns a JWT
@router.post("/auth/register", response_model=TokenResponse)
def register(payload: RegisterRequest):
    with get_session() as session:
        existing = session.scalar(select(User).where(User.username ==
payload.username))
        if existing: raise HTTPException(status_code=400, detail="Username
already exists")
        user =
User(username=payload.username,password_hash=hash_password(payload.passwor
d))
        session.add(user)
        session.commit()
        session.refresh(user)
        token = create_access_token(user.id)
        return TokenResponse(access_token=token)

#verify user credentials and return a JWT
@router.post("/auth/login", response_model=TokenResponse)
def login(payload: LoginRequest):
```

```
    with get_session() as session:
        user = session.scalar(select(User).where(User.username ==
payload.username))
        if not user or not verify_password(payload.password,
user.password_hash): raise HTTPException(status_code=401, detail="Invalid
credentials")
        token = create_access_token(user.id)
        return TokenResponse(access_token=token)

#return user id and username from JWT
@router.get("/auth/me", response_model=MeResponse)
def me(user_id: int = Depends(get_current_user_id)):
    with get_session() as session:
        user = session.get(User, user_id)
        if not user: raise HTTPException(status_code=404, detail="User not
found")
        return MeResponse(id=user.id, username=user.username)

#store users certificate in the app server
@router.post("/certificates/register",
response_model=CertificateRegisterResponse)
def register_certificate(payload: CertificateRegisterRequest, user_id: int
= Depends(get_current_user_id)):
    with get_session() as session:
        owner_id = payload.user_id or user_id
        cert = Certificate(
            user_id=owner_id,
            cert_pem=payload.cert_pem,
            issued_at=datetime.now(timezone.utc).isoformat(),
            expires_at=(datetime.now(timezone.utc) +
timedelta(days=365)).isoformat(),
            revoked=False,
        )
        session.add(cert)
        session.commit()
        session.refresh(cert)
        return CertificateRegisterResponse(cert_id=cert.id)

#create a group and add the group creater as group owner
@router.post("/groups", response_model=GroupResponse)
def create_group(payload: GroupCreateRequest, user_id: int =
Depends(get_current_user_id)):
    with get_session() as session:
        group = Group(name=payload.name, owner_id=user_id)
        session.add(group)
        session.commit()
        session.refresh(group)
        session.add(Membership(group_id=group.id, user_id=user_id,
role="owner"))
        session.commit()
        owner = session.get(User, group.owner_id)
        return GroupResponse(
            id=group.id,
            name=group.name,
```

```
            owner_id=group.owner_id,
            owner_username=owner.username if owner else None,
            members=[user_id],
        )

#return group details
@router.get("/groups/{group_id}", response_model=GroupResponse)
def get_group(group_id: int, user_id: int = Depends(get_current_user_id)):
    with get_session() as session:
        group = session.get(Group, group_id)
        if not group: raise HTTPException(status_code=404, detail="Group
not found")
        members =
session.scalars(select(Membership.user_id).where(Membership.group_id ==
group_id, Membership.active == True)).all()
        if user_id not in members: raise HTTPException(status_code=403,
detail="Not a member")
        owner = session.get(User, group.owner_id)
        return GroupResponse(
            id=group.id,
            name=group.name,
            owner_id=group.owner_id,
            owner_username=owner.username if owner else None,
            members=members,
        )

#return group details but uses group name as look up
@router.get("/groups/by-name/{group_name}", response_model=GroupResponse)
def get_group_by_name(group_name: str, user_id: int =
Depends(get_current_user_id)):
    with get_session() as session:
        group = session.scalar(select(Group).where(Group.name ==
group_name))
        if not group: raise HTTPException(status_code=404, detail="Group
not found")
        members =
session.scalars(select(Membership.user_id).where(Membership.group_id ==
group.id, Membership.active == True)).all()
        if user_id not in members: raise HTTPException(status_code=403,
detail="Not a member")
        owner = session.get(User, group.owner_id)
        return GroupResponse(
            id=group.id,
            name=group.name,
            owner_id=group.owner_id,
            owner_username=owner.username if owner else None,
            members=members,
        )

#list all the groups the user is a member of
@router.get("/groups", response_model=list[GroupListResponse])
def list_my_groups(user_id: int = Depends(get_current_user_id)):
    with get_session() as session:
        memberships =
```

```
session.scalars(select(Membership).where(Membership.user_id == user_id,
Membership.active == True)).all()
        groups = []
        for membership in memberships:
            group = session.get(Group, membership.group_id)
            if not group: continue
            owner = session.get(User, group.owner_id)
            groups.append(
                GroupListResponse(
                    id=group.id,
                    name=group.name,
                    owner_id=group.owner_id,
                    owner_username=owner.username if owner else None,
                )
            )
        return groups

#return group members for the owner
@router.get("/groups/{group_id}/members",
response_model=list[GroupMemberInfo])
def list_group_members(group_id: int, user_id: int =
Depends(get_current_user_id)):
    with get_session() as session:
        group = session.get(Group, group_id)
        if not group: raise HTTPException(status_code=404, detail="Group
not found")
        if group.owner_id != user_id: raise HTTPException(status_code=403,
detail="Only owner can view members")
        memberships =
session.scalars(select(Membership).where(Membership.group_id == group_id,
Membership.active == True)).all()
        members = []
        for membership in memberships:
            user = session.get(User, membership.user_id)
            if not user: continue
            members.append(GroupMemberInfo(id=user.id,
username=user.username, role=membership.role))
        return members

#add a member to a group
@router.post("/groups/{group_id}/members")
def add_member(group_id: int, payload: GroupMemberAddRequest, user_id: int
= Depends(get_current_user_id)):
    with get_session() as session:
        group = session.get(Group, group_id)
        if not group: raise HTTPException(status_code=404, detail="Group
not found")
        if group.owner_id != user_id: raise HTTPException(status_code=403,
detail="Only owner can add members")
        member_id = payload.user_id
        if payload.username and not member_id:
            user = session.scalar(select(User).where(User.username ==
payload.username))
            if not user: raise HTTPException(status_code=404, detail="User
```

```
not found")
            member_id = user.id
        if not member_id: raise HTTPException(status_code=400,
detail="user_id or username required")
        existing = session.scalar(
            select(Membership).where(
                Membership.group_id == group_id,
                Membership.user_id == member_id,
            )
        )
        if existing: existing.active = True
        else: session.add(Membership(group_id=group_id, user_id=member_id,
role="member"))
        session.commit()
        return {"status": "ok"}

#add member to a group based on group name
@router.post("/groups/by-name/{group_name}/members")
def add_member_by_name(group_name: str, payload: GroupMemberAddRequest,
user_id: int = Depends(get_current_user_id)):
    with get_session() as session:
        group = session.scalar(select(Group).where(Group.name ==
group_name))
        if not group: raise HTTPException(status_code=404, detail="Group
not found")
        if group.owner_id != user_id: raise HTTPException(status_code=403,
detail="Only owner can add members")
        member_id = payload.user_id
        if payload.username and not member_id:
            user = session.scalar(select(User).where(User.username ==
payload.username))
            if not user: raise HTTPException(status_code=404, detail="User
not found")
            member_id = user.id
        if not member_id: raise HTTPException(status_code=400,
detail="user_id or username required")
        existing = session.scalar(
            select(Membership).where(
                Membership.group_id == group.id,
                Membership.user_id == member_id,
            )
        )
        if existing: existing.active = True
        else: session.add(Membership(group_id=group.id, user_id=member_id,
role="member"))
        session.commit()
        return {"status": "ok"}

#remove member froma group
@router.delete("/groups/{group_id}/members/{member_id}")
def remove_member(group_id: int, member_id: int, user_id: int =
Depends(get_current_user_id)):
    with get_session() as session:
        group = session.get(Group, group_id)
```

```
        if not group: raise HTTPException(status_code=404, detail="Group
not found")
        if group.owner_id != user_id: raise HTTPException(status_code=403,
detail="Only owner can remove members")
        membership = session.scalar(
            select(Membership).where(
                Membership.group_id == group_id,
                Membership.user_id == member_id,
                Membership.active == True,
            )
        )
        if not membership: raise HTTPException(status_code=404,
detail="Membership not found")
        membership.active = False
        session.commit()
        return {"status": "ok"}

#user leaves a group
@router.post("/groups/{group_id}/leave")
def leave_group(group_id: int, user_id: int =
Depends(get_current_user_id)):
    with get_session() as session:
        group = session.get(Group, group_id)
        if not group: raise HTTPException(status_code=404, detail="Group
not found")
        if group.owner_id == user_id: raise HTTPException(status_code=403,
detail="Owner cannot leave group")
        membership = session.scalar(
            select(Membership).where(
                Membership.group_id == group_id,
                Membership.user_id == user_id,
                Membership.active == True,
            )
        )
        if not membership: raise HTTPException(status_code=404,
detail="Membership not found")
        membership.active = False
        session.commit()
        return {"status": "ok"}

#group owner can delete a group
#removes all posts related to the group
@router.delete("/groups/{group_id}")
def delete_group(group_id: int, user_id: int =
Depends(get_current_user_id)):
    with get_session() as session:
        group = session.get(Group, group_id)
        if not group: raise HTTPException(status_code=404, detail="Group
not found")
        if group.owner_id != user_id: raise HTTPException(status_code=403,
detail="Only owner can delete group")
        version_ids =
select(GroupKeyVersion.id).where(GroupKeyVersion.group_id == group_id)
```

```
session.execute(delete(WrappedKey).where(WrappedKey.group_key_version_id.i
n_(version_ids)))

session.execute(delete(GroupKeyVersion).where(GroupKeyVersion.group_id ==
group_id))
        session.execute(delete(Post).where(Post.group_id == group_id))
        session.execute(delete(Membership).where(Membership.group_id ==
group_id))
        session.delete(group)
        session.commit()
        return {"status": "ok"}

#create a post
@router.post("/groups/{group_id}/posts", response_model=PostResponse)
def create_post(group_id: int, payload: PostCreateRequest, user_id: int =
Depends(get_current_user_id)):
    with get_session() as session:
        membership = session.scalar(
            select(Membership).where(
                Membership.group_id == group_id,
                Membership.user_id == user_id,
                Membership.active == True,
            )
        )
        if not membership: raise HTTPException(status_code=403,
detail="Not a member")
        post = Post(
            group_id=group_id,
            author_id=user_id,
            ciphertext=_b64d(payload.ciphertext),
            nonce=_b64d(payload.nonce),
            auth_tag=_b64d(payload.auth_tag),
            signature=_b64d(payload.signature),
            cert_id=payload.cert_id,
            key_version=payload.key_version,
        )
        session.add(post)
        session.commit()
        session.refresh(post)
        return PostResponse(
            id=post.id,
            group_id=post.group_id,
            author_id=post.author_id,
            author_username=(session.get(User, post.author_id).username if
session.get(User, post.author_id) else None),
            ciphertext=payload.ciphertext,
            nonce=payload.nonce,
            auth_tag=payload.auth_tag,
            signature=payload.signature,
            cert_id=post.cert_id,
            key_version=post.key_version,
        )

#create a post but is identified by groupname
```

```
@router.post("/groups/by-name/{group_name}/posts",
response_model=PostResponse)
def create_post_by_name(group_name: str, payload: PostCreateRequest,
user_id: int = Depends(get_current_user_id)):
    with get_session() as session:
        group = session.scalar(select(Group).where(Group.name ==
group_name))
        if not group: raise HTTPException(status_code=404, detail="Group
not found")
        membership = session.scalar(
            select(Membership).where(
                Membership.group_id == group.id,
                Membership.user_id == user_id,
                Membership.active == True,
            )
        )
        if not membership: raise HTTPException(status_code=403,
detail="Not a member")
        post = Post(
            group_id=group.id,
            author_id=user_id,
            ciphertext=_b64d(payload.ciphertext),
            nonce=_b64d(payload.nonce),
            auth_tag=_b64d(payload.auth_tag),
            signature=_b64d(payload.signature),
            cert_id=payload.cert_id,
            key_version=payload.key_version,
        )
        session.add(post)
        session.commit()
        session.refresh(post)
        author = session.get(User, post.author_id)
        return PostResponse(
            id=post.id,
            group_id=post.group_id,
            author_id=post.author_id,
            author_username=author.username if author else None,
            ciphertext=payload.ciphertext,
            nonce=payload.nonce,
            auth_tag=payload.auth_tag,
            signature=payload.signature,
            cert_id=post.cert_id,
            key_version=post.key_version,
        )

#return all posts from a group
@router.get("/groups/{group_id}/posts", response_model=list[PostResponse])
def list_posts(group_id: int, user_id: int =
Depends(get_current_user_id)):
    with get_session() as session:
        posts = session.scalars(select(Post).where(Post.group_id ==
group_id)).all()
        return [
            PostResponse(
```

```
                id=post.id,
                group_id=post.group_id,
                author_id=post.author_id,
                author_username=(session.get(User,
post.author_id).username if session.get(User, post.author_id) else None),
                ciphertext=_b64(post.ciphertext),
                nonce=_b64(post.nonce),
                auth_tag=_b64(post.auth_tag),
                signature=_b64(post.signature),
                cert_id=post.cert_id,
                key_version=post.key_version,
            )
            for post in posts
        ]

#return all posts
@router.get("/posts", response_model=list[PostResponse])
def list_all_posts(user_id: int = Depends(get_current_user_id)):
    with get_session() as session:
        posts = session.scalars(select(Post)).all()
        return [
            PostResponse(
                id=post.id,
                group_id=post.group_id,
                author_id=post.author_id,
                author_username=(
                    session.get(User, post.author_id).username
                    if session.get(User, post.author_id)
                    else None
                ),
                ciphertext=_b64(post.ciphertext),
                nonce=_b64(post.nonce),
                auth_tag=_b64(post.auth_tag),
                signature=_b64(post.signature),
                cert_id=post.cert_id,
                key_version=post.key_version,
            )
            for post in posts
        ]
```